Arrow Columnar Format

arrow.apache.org/docs/format/Columnar.html

Version: 1.3

The “Arrow Columnar Format” includes a language-agnostic in-memory data structure specification, metadata serialization, and a protocol for serialization and generic data transport.

This document is intended to provide adequate detail to create a new implementation of the columnar format without the aid of an existing implementation. We utilize Google’s Flatbuffers project for metadata serialization, so it will be necessary to refer to the project’s Flatbuffers protocol definition files while reading this document.

The columnar format has some key features:

- Data adjacency for sequential access (scans)
- O(1) (constant-time) random access
- SIMD and vectorization-friendly
- Relocatable without “pointer swizzling”, allowing for true zero-copy access in shared memory

The Arrow columnar format provides analytical performance and data locality guarantees in exchange for comparatively more expensive mutation operations. This document is concerned only with in-memory data representation and serialization details; issues such as coordinating mutation of data structures are left to be handled by implementations.

Terminology

Since different projects have used different words to describe various concepts, here is a small glossary to help disambiguate.

- **Array** or **Vector**: a sequence of values with known length all having the same type. These terms are used interchangeably in different Arrow implementations, but we use “array” in this document.
- **Slot**: a single logical value in an array of some particular data type
- **Buffer** or **Contiguous memory region**: a sequential virtual address space with a given length. Any byte can be reached via a single pointer offset less than the region’s length.

- **Physical Layout**: The underlying memory layout for an array without taking into account any value semantics. For example, a 32-bit signed integer array and 32-bit floating point array have the same layout.
- **Parent** and **child arrays**: names to express relationships between physical value arrays in a nested type structure. For example, a `List<T>`-type parent array has a T-type array as its child (see more on lists below).
- **Primitive type**: a data type having no child types. This includes such types as fixed bit-width, variable-size binary, and null types.
- **Nested type**: a data type whose full structure depends on one or more other child types. Two fully-specified nested types are equal if and only if their child types are equal. For example, `List<U>` is distinct from `List<V>` iff U and V are different types.
- **Logical type**: An application-facing semantic value type that is implemented using some physical layout. For example, Decimal values are stored as 16 bytes in a fixed-size binary layout. Similarly, strings can be stored as `List<1-byte>`. A timestamp may be stored as 64-bit fixed-size layout.

Physical Memory Layout

Arrays are defined by a few pieces of metadata and data:

- A logical data type.
- A sequence of buffers.
- A length as a 64-bit signed integer. Implementations are permitted to be limited to 32-bit lengths, see more on this below.
- A null count as a 64-bit signed integer.
- An optional **dictionary**, for dictionary-encoded arrays.

Nested arrays additionally have a sequence of one or more sets of these items, called the **child arrays**.

Each logical data type has a well-defined physical layout. Here are the different physical layouts defined by Arrow:

- **Primitive (fixed-size)**: a sequence of values each having the same byte or bit width
- **Variable-size Binary**: a sequence of values each having a variable byte length. Two variants of this layout are supported using 32-bit and 64-bit length encoding.

- **Fixed-size List**: a nested layout where each value has the same number of elements taken from a child data type.
- **Variable-size List**: a nested layout where each value is a variable-length sequence of values taken from a child data type. Two variants of this layout are supported using 32-bit and 64-bit length encoding.
- **Struct**: a nested layout consisting of a collection of named child **fields** each having the same length but possibly different types.
- **Sparse** and **Dense Union**: a nested layout representing a sequence of values, each of which can have type chosen from a collection of child array types.
- **Dictionary-Encoded**: a layout consisting of a sequence of integers (any bit-width) which represent indexes into a dictionary which could be of any type.
- **Run-End Encoded (REE)**: a nested layout consisting of two child arrays, one representing values, and one representing the logical index where the run of a corresponding value ends.
- **Null**: a sequence of all null values, having null logical type

The Arrow columnar memory layout only applies to *data* and not *metadata*. Implementations are free to represent metadata in-memory in whichever form is convenient for them. We handle metadata **serialization** in an implementation-independent way using Flatbuffers, detailed below.

Buffer Alignment and Padding

Implementations are recommended to allocate memory on aligned addresses (multiple of 8- or 64-bytes) and pad (overallocate) to a length that is a multiple of 8 or 64 bytes. When serializing Arrow data for interprocess communication, these alignment and padding requirements are enforced. If possible, we suggest that you prefer using 64-byte alignment and padding. Unless otherwise noted, padded bytes do not need to have a specific value.

The alignment requirement follows best practices for optimized memory access:

- Elements in numeric arrays will be guaranteed to be retrieved via aligned access.
- On some architectures alignment can help limit partially used cache lines.

The recommendation for 64 byte alignment comes from the Intel performance guide that recommends alignment of memory to match SIMD register width. The specific padding length was chosen because it matches the largest SIMD instruction registers available on widely deployed x86 architecture (Intel AVX-512).

The recommended padding of 64 bytes allows for using SIMD instructions consistently in loops without additional conditional checks. This should allow for simpler, efficient and CPU cache-friendly code. In other words, we can load the entire 64-byte buffer into a 512-bit wide SIMD register and get data-level parallelism on all the columnar values packed into the 64-byte buffer. Guaranteed padding can also allow certain compilers to generate more optimized code directly (e.g. One can safely use Intel's `-qopt-assume-safe-padding`).

Array lengths

Array lengths are represented in the Arrow metadata as a 64-bit signed integer. An implementation of Arrow is considered valid even if it only supports lengths up to the maximum 32-bit signed integer, though. If using Arrow in a multi-language environment, we recommend limiting lengths to $2^{31} - 1$ elements or less. Larger data sets can be represented using multiple array chunks.

Null count

The number of null value slots is a property of the physical array and considered part of the data structure. The null count is represented in the Arrow metadata as a 64-bit signed integer, as it may be as large as the array length.

Validity bitmaps

Any value in an array may be semantically null, whether primitive or nested type.

All array types, with the exception of union types (more on these later), utilize a dedicated memory buffer, known as the validity (or "null") bitmap, to encode the nullness or non-nullness of each value slot. The validity bitmap must be large enough to have at least 1 bit for each array slot.

Whether any array slot is valid (non-null) is encoded in the respective bits of this bitmap. A 1 (set bit) for index `j` indicates that the value is not null, while a 0 (bit not set) indicates that it is null. Bitmaps are to be initialized to be all unset at allocation time (this includes padding):

```
is_valid[j] -> bitmap[j / 8] & (1 << (j % 8))
```

We use least-significant bit (LSB) numbering (also known as bit-endianness). This means that within a group of 8 bits, we read right-to-left:

```
values = [0, 1, null, 2, null, 3]

bitmap
j mod 8   7  6  5  4  3  2  1  0
          0  0  1  0  1  0  1  1
```

Arrays having a 0 null count may choose to not allocate the validity bitmap; how this is represented depends on the implementation (for example, a C++ implementation may represent such an “absent” validity bitmap using a NULL pointer). Implementations may choose to always allocate a validity bitmap anyway as a matter of convenience. Consumers of Arrow arrays should be ready to handle those two possibilities.

Nested type arrays (except for union types as noted above) have their own top-level validity bitmap and null count, regardless of the null count and valid bits of their child arrays.

Array slots which are null are not required to have a particular value; any “masked” memory can have any value and need not be zeroed, though implementations frequently choose to zero memory for null values.

Fixed-size Primitive Layout

A primitive value array represents an array of values each having the same physical slot width typically measured in bytes, though the spec also provides for bit-packed types (e.g. boolean values encoded in bits).

Internally, the array contains a contiguous memory buffer whose total size is at least as large as the slot width multiplied by the array length. For bit-packed types, the size is rounded up to the nearest byte.

The associated validity bitmap is contiguously allocated (as described above) but does not need to be adjacent in memory to the values buffer.

Example Layout: Int32 Array

For example a primitive array of int32s:

```
[1, null, 2, 4, 8]
```

Would look like:

```
* Length: 5, Null count: 1
* Validity bitmap buffer:

  |Byte 0 (validity bitmap) | Bytes 1-63            |
  |-------------------------|-----------------------|
  | 00011101                | 0 (padding)           |

* Value Buffer:

  |Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | Bytes 12-15 | Bytes 16-19 | Bytes 20-63
|
  |------------|-------------|-------------|-------------|-------------|-------------
|
  | 1          | unspecified | 2           | 4           | 8           | unspecified
|
```

Example Layout: Non-null int32 Array

`[1, 2, 3, 4, 8]` has two possible layouts:

```
* Length: 5, Null count: 0
* Validity bitmap buffer:

  | Byte 0 (validity bitmap) | Bytes 1-63            |
  |--------------------------|-----------------------|
  | 00011111                 | 0 (padding)           |

* Value Buffer:

  |Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | bytes 12-15 | bytes 16-19 | Bytes 20-63
|
  |------------|-------------|-------------|-------------|-------------|-------------
|
  | 1          | 2           | 3           | 4           | 8           | unspecified
|
```

or with the bitmap elided:

```
* Length 5, Null count: 0
* Validity bitmap buffer: Not required
* Value Buffer:

  |Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | bytes 12-15 | bytes 16-19 | Bytes 20-63
|
  |------------|-------------|-------------|-------------|-------------|-------------
|
  | 1          | 2           | 3           | 4           | 8           | unspecified
|
```

Variable-size Binary Layout

Each value in this layout consists of 0 or more bytes. While primitive arrays have a single values buffer, variable-size binary have an **offsets** buffer and **data** buffer.

The offsets buffer contains *length + 1* signed integers (either 32-bit or 64-bit, depending on the logical type), which encode the start position of each slot in the data buffer. The length of the value in each slot is computed using the difference between the offset at that slot's index and the subsequent offset. For example, the position and length of slot j is computed as:

```
slot_position = offsets[j]
slot_length = offsets[j + 1] - offsets[j]  // (for 0 <= j < length)
```

It should be noted that a null value may have a positive slot length. That is, a null value may occupy a **non-empty** memory space in the data buffer. When this is true, the content of the corresponding memory space is undefined.

Offsets must be monotonically increasing, that is `offsets[j+1] >= offsets[j]` for `0 <= j < length`, even for null slots. This property ensures the location for all values is valid and well defined.

Generally the first slot in the offsets array is 0, and the last slot is the length of the values array. When serializing this layout, we recommend normalizing the offsets to start at 0.

Example Layout: ``VarBinary``

`['joe', null, null, 'mark']`

will be represented as follows:

```
* Length: 4, Null count: 2
* Validity bitmap buffer:

  | Byte 0 (validity bitmap) | Bytes 1-63            |
  |--------------------------|-----------------------|
  | 00001001                 | 0 (padding)           |

* Offsets buffer:

  | Bytes 0-19     | Bytes 20-63           |
  |----------------|-----------------------|
  | 0, 3, 3, 3, 7  | unspecified           |

 * Value buffer:

  | Bytes 0-6      | Bytes 7-63           |
  |----------------|----------------------|
  | joemark        | unspecified          |
```

Variable-size List Layout

List is a nested type which is semantically similar to variable-size binary. It is defined by two buffers, a validity bitmap and an offsets buffer, and a child array. The offsets are the same as in the variable-size binary case, and both 32-bit and 64-bit signed integer offsets are supported options for the offsets. Rather than referencing an additional data buffer, instead these offsets reference the child array.

Similar to the layout of variable-size binary, a null value may correspond to a **non-empty** segment in the child array. When this is true, the content of the corresponding segment can be arbitrary.

A list type is specified like `List<T>`, where `T` is any type (primitive or nested). In these examples we use 32-bit offsets where the 64-bit offset version would be denoted by `LargeList<T>`.

Example Layout: ``List<Int8>`` Array

We illustrate an example of `List<Int8>` with length 4 having values:

```
[[12, -7, 25], null, [0, -127, 127, 50], []]
```

will have the following representation:

```
* Length: 4, Null count: 1
* Validity bitmap buffer:

  | Byte 0 (validity bitmap) | Bytes 1-63            |
  |--------------------------|-----------------------|
  | 00001101                 | 0 (padding)           |

* Offsets buffer (int32)

  | Bytes 0-3  | Bytes 4-7   | Bytes 8-11  | Bytes 12-15 | Bytes 16-19 | Bytes 20-63
|
  |------------|-------------|-------------|-------------|-------------|-------------
|
  | 0          | 3           | 3           | 7           | 7           | unspecified
|

* Values array (Int8array):
  * Length: 7,  Null count: 0
  * Validity bitmap buffer: Not required
  * Values buffer (int8)

    | Bytes 0-6                    | Bytes 7-63  |
    |------------------------------|-------------|
    | 12, -7, 25, 0, -127, 127, 50 | unspecified |
```

Example Layout: ``List<List<Int8>>``

`[[[1, 2], [3, 4]], [[5, 6, 7], null, [8]], [[9, 10]]]`

will be represented as follows:

```
* Length 3
* Nulls count: 0
* Validity bitmap buffer: Not required
* Offsets buffer (int32)

  | Bytes 0-3  | Bytes 4-7  | Bytes 8-11 | Bytes 12-15 | Bytes 16-63 |
  |------------|------------|------------|-------------|-------------|
  | 0          |  2         |  5         |  6          | unspecified |

* Values array (`List<Int8>`)
  * Length: 6, Null count: 1
  * Validity bitmap buffer:

    | Byte 0 (validity bitmap) | Bytes 1-63  |
    |--------------------------|-------------|
    | 00110111                 | 0 (padding) |

  * Offsets buffer (int32)

    | Bytes 0-27           | Bytes 28-63 |
    |----------------------|-------------|
    | 0, 2, 4, 7, 7, 8, 10 | unspecified |

  * Values array (Int8):
    * Length: 10, Null count: 0
    * Validity bitmap buffer: Not required

      | Bytes 0-9                     | Bytes 10-63 |
      |-------------------------------|-------------|
      | 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 | unspecified |
```

Fixed-Size List Layout

Fixed-Size List is a nested type in which each array slot contains a fixed-size sequence of values all having the same type.

A fixed size list type is specified like `FixedSizeList<T>[N]`, where `T` is any type (primitive or nested) and `N` is a 32-bit signed integer representing the length of the lists.

A fixed size list array is represented by a values array, which is a child array of type T. T may also be a nested type. The value in slot `j` of a fixed size list array is stored in an `N`-long slice of the values array, starting at an offset of `j * N`.

Example Layout: ``FixedSizeList<byte>[4]`` Array

Here we illustrate `FixedSizeList<byte>[4]`.

For an array of length 4 with respective values:

```
[[192, 168, 0, 12], null, [192, 168, 0, 25], [192, 168, 0, 1]]
```

will have the following representation:

```
* Length: 4, Null count: 1
* Validity bitmap buffer:

  | Byte 0 (validity bitmap) | Bytes 1-63            |
  |--------------------------|-----------------------|
  | 00001101                 | 0 (padding)           |

* Values array (byte array):
  * Length: 16,  Null count: 0
  * validity bitmap buffer: Not required

    | Bytes 0-3       | Bytes 4-7   | Bytes 8-15                      |
    |-----------------|-------------|---------------------------------|
    | 192, 168, 0, 12 | unspecified | 192, 168, 0, 25, 192, 168, 0, 1 |
```

Struct Layout

A struct is a nested type parameterized by an ordered sequence of types (which can all be distinct), called its fields. Each field must have a UTF8-encoded name, and these field names are part of the type metadata.

Physically, a struct array has one child array for each field. The child arrays are independent and need not be adjacent to each other in memory. A struct array also has a validity bitmap to encode top-level validity information.

For example, the struct (field names shown here as strings for illustration purposes):

```
Struct <
  name: VarBinary
  age: Int32
>
```

has two child arrays, one `VarBinary` array (using variable-size binary layout) and one 4-byte primitive value array having `Int32` logical type.

Example Layout: ``Struct<VarBinary, Int32>``

The layout for `[{'joe', 1}, {null, 2}, null, {'mark', 4}]` would be:

```
* Length: 4, Null count: 1
* Validity bitmap buffer:

  |Byte 0 (validity bitmap) | Bytes 1-63            |
  |-------------------------|-----------------------|
  | 00001011                | 0 (padding)           |

* Children arrays:
  * field-0 array (`VarBinary`):
    * Length: 4, Null count: 2
    * Validity bitmap buffer:

      | Byte 0 (validity bitmap) | Bytes 1-63            |
      |--------------------------|-----------------------|
      | 00001001                 | 0 (padding)           |

    * Offsets buffer:

      | Bytes 0-19     | Bytes 20-63           |
      |----------------|-----------------------|
      | 0, 3, 3, 3, 7  | unspecified           |

     * Value buffer:

      | Bytes 0-6      | Bytes 7-63            |
      |----------------|-----------------------|
      | joemark        | unspecified           |

  * field-1 array (int32 array):
    * Length: 4, Null count: 1
    * Validity bitmap buffer:

      | Byte 0 (validity bitmap) | Bytes 1-63            |
      |--------------------------|-----------------------|
      | 00001011                 | 0 (padding)           |

    * Value Buffer:

      |Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | Bytes 12-15 | Bytes 16-63 |
      |------------|-------------|-------------|-------------|-------------|
      | 1          | 2           | unspecified | 4           | unspecified |
```

Struct Validity

A struct array has its own validity bitmap that is independent of its child arrays' validity bitmaps. The validity bitmap for the struct array might indicate a null when one or more of its child arrays has a non-null value in its corresponding slot; or conversely, a child array might have a null in its validity bitmap while the struct array's validity bitmap shows a non-null value.

Therefore, to know whether a particular child entry is valid, one must take the logical AND of the corresponding bits in the two validity bitmaps (the struct array's and the child array's).

This is illustrated in the example above, the child arrays have valid entries for the null struct but they are "hidden" by the struct array's validity bitmap. However, when treated independently, corresponding entries of the children array will be non-null.

Union Layout

A union is defined by an ordered sequence of types; each slot in the union can have a value chosen from these types. The types are named like a struct's fields, and the names are part of the type metadata.

Unlike other data types, unions do not have their own validity bitmap. Instead, the nullness of each slot is determined exclusively by the child arrays which are composed to create the union.

We define two distinct union types, "dense" and "sparse", that are optimized for different use cases.

Dense Union

Dense union represents a mixed-type array with 5 bytes of overhead for each value. Its physical layout is as follows:

- One child array for each type
- Types buffer: A buffer of 8-bit signed integers. Each type in the union has a corresponding type id whose values are found in this buffer. A union with more than 127 possible types can be modeled as a union of unions.
- Offsets buffer: A buffer of signed Int32 values indicating the relative offset into the respective child array for the type in a given slot. The respective offsets for each child value array must be in order / increasing.

Example Layout: ``DenseUnion<f: Float32, i: Int32>``

For the union array:

```
[{f=1.2}, null, {f=3.4}, {i=5}]
```

will have the following layout:

```
* Length: 4, Null count: 0
* Types buffer:

  |Byte 0   | Byte 1      | Byte 2   | Byte 3   | Bytes 4-63  |
  |---------|-------------|----------|----------|-------------|
  | 0       | 0           | 0        | 1        | unspecified |

* Offset buffer:

  |Bytes 0-3 | Bytes 4-7   | Bytes 8-11 | Bytes 12-15 | Bytes 16-63 |
  |----------|-------------|------------|-------------|-------------|
  | 0        | 1           | 2          | 0           | unspecified |

* Children arrays:
  * Field-0 array (f: Float32):
    * Length: 3, Null count: 1
    * Validity bitmap buffer: 00000101

    * Value Buffer:

      | Bytes 0-11     | Bytes 12-63  |
      |----------------|-------------|
      | 1.2, null, 3.4 | unspecified |


  * Field-1 array (i: Int32):
    * Length: 1, Null count: 0
    * Validity bitmap buffer: Not required

    * Value Buffer:

      | Bytes 0-3 | Bytes 4-63  |
      |-----------|-------------|
      | 5         | unspecified |
```

Sparse Union

A sparse union has the same structure as a dense union, with the omission of the offsets array. In this case, the child arrays are each equal in length to the length of the union.

While a sparse union may use significantly more space compared with a dense union, it has some advantages that may be desirable in certain use cases:

- A sparse union is more amenable to vectorized expression evaluation in some use cases.
- Equal-length arrays can be interpreted as a union by only defining the types array.

Example layout: ``SparseUnion<i: Int32, f: Float32, s: VarBinary>``

For the union array:

```
[{i=5}, {f=1.2}, {s='joe'}, {f=3.4}, {i=4}, {s='mark'}]
```

will have the following layout:

```
* Length: 6, Null count: 0
* Types buffer:

 | Byte 0     | Byte 1      | Byte 2      | Byte 3      | Byte 4      | Byte 5
| Bytes  6-63           |
 |------------|-------------|-------------|-------------|-------------|--------------
|-----------------------|
 | 0          | 1           | 2           | 1           | 0           | 2
| unspecified (padding) |

* Children arrays:

  * i (Int32):
    * Length: 6, Null count: 4
    * Validity bitmap buffer:

      |Byte 0 (validity bitmap) | Bytes 1-63            |
      |-------------------------|-----------------------|
      |00010001                 | 0 (padding)           |

    * Value buffer:

      |Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | Bytes 12-15 | Bytes 16-19 | Bytes
20-23  | Bytes 24-63           |
      |------------|-------------|-------------|-------------|-------------|---------
-----|-----------------------|
      | 5          | unspecified | unspecified | unspecified | 4           |
unspecified | unspecified (padding) |

  * f (Float32):
    * Length: 6, Null count: 4
    * Validity bitmap buffer:

      |Byte 0 (validity bitmap) | Bytes 1-63            |
      |-------------------------|-----------------------|
      | 00001010                | 0 (padding)           |

    * Value buffer:

      |Bytes 0-3    | Bytes 4-7   | Bytes 8-11  | Bytes 12-15 | Bytes 16-19 | Bytes
20-23  | Bytes 24-63           |
      |-------------|-------------|-------------|-------------|-------------|--------
------|-----------------------|
      | unspecified |  1.2        | unspecified | 3.4         | unspecified |
unspecified | unspecified (padding) |

  * s (`VarBinary`)
    * Length: 6, Null count: 4
    * Validity bitmap buffer:

      | Byte 0 (validity bitmap) | Bytes 1-63            |
      |--------------------------|-----------------------|
```

```
      | 00100100                 | 0 (padding)           |

    * Offsets buffer (Int32)

      | Bytes 0-3  | Bytes 4-7   | Bytes 8-11  | Bytes 12-15 | Bytes 16-19 | Bytes
20-23 | Bytes 24-27 | Bytes 28-63 |
      |------------|-------------|-------------|-------------|-------------|---------
----|-------------|-------------|
      | 0          | 0           | 0           | 3           | 3           | 3
| 7           | unspecified |

    * Values buffer:

      | Bytes 0-6  | Bytes 7-63            |
      |------------|-----------------------|
      | joemark    | unspecified (padding) |
```

Only the slot in the array corresponding to the type index is considered. All “unselected” values are ignored and could be any semantically correct array value.

Null Layout

We provide a simplified memory-efficient layout for the Null data type where all values are null. In this case no memory buffers are allocated.

Dictionary-encoded Layout

Dictionary encoding is a data representation technique to represent values by integers referencing a **dictionary** usually consisting of unique values. It can be effective when you have data with many repeated values.

Any array can be dictionary-encoded. The dictionary is stored as an optional property of an array. When a field is dictionary encoded, the values are represented by an array of non-negative integers representing the index of the value in the dictionary. The memory layout for a dictionary-encoded array is the same as that of a primitive integer layout. The dictionary is handled as a separate columnar array with its own respective layout.

As an example, you could have the following data:

```
type: VarBinary

['foo', 'bar', 'foo', 'bar', null, 'baz']
```

In dictionary-encoded form, this could appear as:

```
data VarBinary (dictionary-encoded)
   index_type: Int32
   values: [0, 1, 0, 1, null, 2]

dictionary
   type: VarBinary
   values: ['foo', 'bar', 'baz']
```

Note that a dictionary is permitted to contain duplicate values or nulls:

```
data VarBinary (dictionary-encoded)
   index_type: Int32
   values: [0, 1, 3, 1, 4, 2]

dictionary
   type: VarBinary
   values: ['foo', 'bar', 'baz', 'foo', null]
```

The null count of such arrays is dictated only by the validity bitmap of its indices, irrespective of any null values in the dictionary.

Since unsigned integers can be more difficult to work with in some cases (e.g. in the JVM), we recommend preferring signed integers over unsigned integers for representing dictionary indices. Additionally, we recommend avoiding using 64-bit unsigned integer indices unless they are required by an application.

We discuss dictionary encoding as it relates to serialization further below.

Run-End Encoded Layout

Run-end encoding (REE) is a variation of run-length encoding (RLE). These encodings are well-suited for representing data containing sequences of the same value, called runs. In run-end encoding, each run is represented as a value and an integer giving the index in the array where the run ends.

Any array can be run-end encoded. A run-end encoded array has no buffers by itself, but has two child arrays. The first child array, called the run ends array, holds either 16, 32, or 64-bit signed integers. The actual values of each run are held in the second child array. For the purposes of determining field names and schemas, these child arrays are prescribed the standard names of **run_ends** and **values** respectively.

The values in the first child array represent the accumulated length of all runs from the first to the current one, i.e. the logical index where the current run ends. This allows relatively efficient random access from a logical index using binary search. The length of an individual run can be determined by subtracting two adjacent values. (Contrast this with run-length encoding, in which the lengths of the runs are represented directly, and in which random access is less efficient.)

Note

Because the `run_ends` child array cannot have nulls, it's reasonable to consider why the `run_ends` are a child array instead of just a buffer, like the offsets for a Variable-size List Layout. This layout was considered, but it was decided to use the child arrays.

Child arrays allow us to keep the "logical length" (the decoded length) associated with the parent array and the "physical length" (the number of run ends) associated with the child arrays. If `run_ends` was a buffer in the parent array then the size of the buffer would be unrelated to the length of the array and this would be confusing.

A run must have have a length of at least 1. This means the values in the run ends array all are positive and in strictly ascending order. A run end cannot be null.

The REE parent has no validity bitmap, and it's null count field should always be 0. Null values are encoded as runs with the value null.

As an example, you could have the following data:

```
type: Float32
[1.0, 1.0, 1.0, 1.0, null, null, 2.0]
```

In Run-end-encoded form, this could appear as:

```
* Length: 7, Null count: 0
* Child Arrays:

  * run_ends (Int32):
    * Length: 3, Null count: 0 (Run Ends cannot be null)
    * Validity bitmap buffer: Not required (if it exists, it should be all 1s)
    * Values buffer

      | Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | Bytes 12-63           |
      |-------------|-------------|-------------|-----------------------|
      | 4           | 6           | 7           | unspecified (padding) |

  * values (Float32):
    * Length: 3, Null count: 1
    * Validity bitmap buffer:

      | Byte 0 (validity bitmap) | Bytes 1-63            |
      |--------------------------|-----------------------|
      | 00000101                 | 0 (padding)           |

    * Values buffer

      | Bytes 0-3   | Bytes 4-7   | Bytes 8-11  | Bytes 12-63           |
      |-------------|-------------|-------------|-----------------------|
      | 1.0         | unspecified | 2.0         | unspecified (padding) |
```

Buffer Listing for Each Layout

For the avoidance of ambiguity, we provide listing the order and type of memory buffers for each layout.

Layout Type	Buffer 0	Buffer 1	Buffer 2
Primitive	validity	data	
Variable Binary	validity	offsets	data
List	validity	offsets	
Fixed-size List	validity		
Struct	validity		
Sparse Union	type ids		
Dense Union	type ids	offsets	
Null			
Dictionary-encoded	validity	data (indices)	
Run-end encoded			

Buffer Layouts

Logical Types

The Schema.fbs defines built-in logical types supported by the Arrow columnar format. Each logical type uses one of the above physical layouts. Nested logical types may have different physical layouts depending on the particular realization of the type.

We do not go into detail about the logical types definitions in this document as we consider Schema.fbs to be authoritative.

Serialization and Interprocess Communication (IPC)

The primitive unit of serialized data in the columnar format is the “record batch”. Semantically, a record batch is an ordered collection of arrays, known as its **fields**, each having the same length as one another but potentially different data types. A record batch’s field names and types collectively form the batch’s **schema**.

In this section we define a protocol for serializing record batches into a stream of binary payloads and reconstructing record batches from these payloads without need for memory copying.

The columnar IPC protocol utilizes a one-way stream of binary messages of these types:

- Schema
- RecordBatch
- DictionaryBatch

We specify a so-called *encapsulated IPC message* format which includes a serialized Flatbuffer type along with an optional message body. We define this message format before describing how to serialize each constituent IPC message type.

Encapsulated message format

For simple streaming and file-based serialization, we define a “encapsulated” message format for interprocess communication. Such messages can be “deserialized” into in-memory Arrow array objects by examining only the message metadata without any need to copy or move any of the actual data.

The encapsulated binary message format is as follows:

- A 32-bit continuation indicator. The value `0xFFFFFFFF` indicates a valid message. This component was introduced in version 0.15.0 in part to address the 8-byte alignment requirement of Flatbuffers
- A 32-bit little-endian length prefix indicating the metadata size
- The message metadata as using the `Message` type defined in Message.fbs
- Padding bytes to an 8-byte boundary
- The message body, whose length must be a multiple of 8 bytes

Schematically, we have:

```
<continuation: 0xFFFFFFFF>
<metadata_size: int32>
<metadata_flatbuffer: bytes>
<padding>
<message body>
```

The complete serialized message must be a multiple of 8 bytes so that messages can be relocated between streams. Otherwise the amount of padding between the metadata and the message body could be non-deterministic.

The `metadata_size` includes the size of the `Message` plus padding. The `metadata_flatbuffer` contains a serialized `Message` Flatbuffer value, which internally includes:

- A version number
- A particular message value (one of `Schema`, `RecordBatch`, or `DictionaryBatch`)
- The size of the message body
- A `custom_metadata` field for any application-supplied metadata

When read from an input stream, generally the `Message` metadata is initially parsed and validated to obtain the body size. Then the body can be read.

Schema message

The Flatbuffers files Schema.fbs contains the definitions for all built-in logical data types and the `Schema` metadata type which represents the schema of a given record batch. A schema consists of an ordered sequence of fields, each having a name and type. A serialized `Schema` does not contain any data buffers, only type metadata.

The `Field` Flatbuffers type contains the metadata for a single array. This includes:

- The field's name
- The field's logical type
- Whether the field is semantically nullable. While this has no bearing on the array's physical layout, many systems distinguish nullable and non-nullable fields and we want to allow them to preserve this metadata to enable faithful schema round trips.
- A collection of child `Field` values, for nested types
- A `dictionary` property indicating whether the field is dictionary-encoded or not. If it is, a dictionary "id" is assigned to allow matching a subsequent dictionary IPC message with the appropriate field.

We additionally provide both schema-level and field-level `custom_metadata` attributes allowing for systems to insert their own application defined metadata to customize behavior.

RecordBatch message

A RecordBatch message contains the actual data buffers corresponding to the physical memory layout determined by a schema. The metadata for this message provides the location and size of each buffer, permitting Array data structures to be reconstructed using pointer arithmetic and thus no memory copying.

The serialized form of the record batch is the following:

- The `data header`, defined as the `RecordBatch` type in Message.fbs.
- The `body`, a flat sequence of memory buffers written end-to-end with appropriate padding to ensure a minimum of 8-byte alignment

The data header contains the following:

- The length and null count for each flattened field in the record batch
- The memory offset and length of each constituent `Buffer` in the record batch's body

Fields and buffers are flattened by a pre-order depth-first traversal of the fields in the record batch. For example, let's consider the schema

```
col1: Struct<a: Int32, b: List<item: Int64>, c: Float64>
col2: Utf8
```

The flattened version of this is:

```
FieldNode 0: Struct name='col1'
FieldNode 1: Int32 name='a'
FieldNode 2: List name='b'
FieldNode 3: Int64 name='item'
FieldNode 4: Float64 name='c'
FieldNode 5: Utf8 name='col2'
```

For the buffers produced, we would have the following (refer to the table above):

```
buffer 0: field 0 validity
buffer 1: field 1 validity
buffer 2: field 1 values
buffer 3: field 2 validity
buffer 4: field 2 offsets
buffer 5: field 3 validity
buffer 6: field 3 values
buffer 7: field 4 validity
buffer 8: field 4 values
buffer 9: field 5 validity
buffer 10: field 5 offsets
buffer 11: field 5 data
```

The `Buffer` Flatbuffers value describes the location and size of a piece of memory. Generally these are interpreted relative to the **encapsulated message format** defined below.

The `size` field of `Buffer` is not required to account for padding bytes. Since this metadata can be used to communicate in-memory pointer addresses between libraries, it is recommended to set `size` to the actual memory size rather than the padded size.

Byte Order (Endianness)

The Arrow format is little endian by default.

Serialized Schema metadata has an endianness field indicating endianness of RecordBatches. Typically this is the endianness of the system where the RecordBatch was generated. The main use case is exchanging RecordBatches between systems with the same Endianness. At first we will return an error when trying to read a Schema with an endianness that does not match the underlying system. The reference implementation is focused on Little Endian and provides tests for it. Eventually we may provide automatic conversion via byte swapping.

IPC Streaming Format

We provide a streaming protocol or “format” for record batches. It is presented as a sequence of encapsulated messages, each of which follows the format above. The schema comes first in the stream, and it is the same for all of the record batches that follow. If any fields in the schema are dictionary-encoded, one or more `DictionaryBatch` messages will be included. `DictionaryBatch` and `RecordBatch` messages may be interleaved, but before any dictionary key is used in a `RecordBatch` it should be defined in a `DictionaryBatch`.

```
<SCHEMA>
<DICTIONARY 0>
...
<DICTIONARY k - 1>
<RECORD BATCH 0>
...
<DICTIONARY x DELTA>
...
<DICTIONARY y DELTA>
...
<RECORD BATCH n - 1>
<EOS [optional]: 0xFFFFFFFF 0x00000000>
```

Note

An edge-case for interleaved dictionary and record batches occurs when the record batches contain dictionary encoded arrays that are completely null. In this case, the dictionary for the encoded column might appear after the first record batch.

When a stream reader implementation is reading a stream, after each message, it may read the next 8 bytes to determine both if the stream continues and the size of the message metadata that follows. Once the message flatbuffer is read, you can then read the message body.

The stream writer can signal end-of-stream (EOS) either by writing 8 bytes containing the 4-byte continuation indicator (`0xFFFFFFFF`) followed by 0 metadata length (`0x00000000`) or closing the stream interface. We recommend the ".arrows" file extension for the streaming format although in many cases these streams will not ever be stored as files.

IPC File Format

We define a "file format" supporting random access that is an extension of the stream format. The file starts and ends with a magic string `ARROW1` (plus padding). What follows in the file is identical to the stream format. At the end of the file, we write a *footer* containing a redundant copy of the schema (which is a part of the streaming format) plus memory offsets and sizes for each of the data blocks in the file. This enables random access to any record batch in the file. See File.fbs for the precise details of the file footer.

Schematically we have:

```
<magic number "ARROW1">
<empty padding bytes [to 8 byte boundary]>
<STREAMING FORMAT with EOS>
<FOOTER>
<FOOTER SIZE: int32>
<magic number "ARROW1">
```

In the file format, there is no requirement that dictionary keys should be defined in a `DictionaryBatch` before they are used in a `RecordBatch`, as long as the keys are defined somewhere in the file. Further more, it is invalid to have more than one **non-delta** dictionary batch per dictionary ID (i.e. dictionary replacement is not supported). Delta dictionaries are applied in the order they appear in the file footer. We recommend the ".arrow" extension for files created with this format. Note that files created with this format are sometimes called "Feather V2" or with the ".feather" extension, the name and the extension derived from "Feather (V1)", which was a proof of concept early in the Arrow project for language-agnostic fast data frame storage for Python (pandas) and R.

Dictionary Messages

Dictionaries are written in the stream and file formats as a sequence of record batches, each having a single field. The complete semantic schema for a sequence of record batches, therefore, consists of the schema along with all of the dictionaries. The dictionary types are found in the schema, so it is necessary to read the schema to first determine the dictionary types so that the dictionaries can be properly interpreted:

```
table DictionaryBatch {
  id: long;
  data: RecordBatch;
  isDelta: boolean = false;
}
```

The dictionary `id` in the message metadata can be referenced one or more times in the schema, so that dictionaries can even be used for multiple fields. See the Dictionary-encoded Layout section for more about the semantics of dictionary-encoded data.

The dictionary `isDelta` flag allows existing dictionaries to be expanded for future record batch materializations. A dictionary batch with `isDelta` set indicates that its vector should be concatenated with those of any previous batches with the same `id`. In a stream which encodes one column, the list of strings `["A", "B", "C", "B", "D", "C", "E", "A"]`, with a delta dictionary batch could take the form:

```
<SCHEMA>
<DICTIONARY 0>
(0) "A"
(1) "B"
(2) "C"

<RECORD BATCH 0>
0
1
2
1

<DICTIONARY 0 DELTA>
(3) "D"
(4) "E"

<RECORD BATCH 1>
3
2
4
0
EOS
```

Alternatively, if `isDelta` is set to false, then the dictionary replaces the existing dictionary for the same ID. Using the same example as above, an alternate encoding could be:

```
<SCHEMA>
<DICTIONARY 0>
(0) "A"
(1) "B"
(2) "C"

<RECORD BATCH 0>
0
1
2
1

<DICTIONARY 0>
(0) "A"
(1) "C"
(2) "D"
(3) "E"

<RECORD BATCH 1>
2
1
3
0
EOS
```

Custom Application Metadata

We provide a `custom_metadata` field at three levels to provide a mechanism for developers to pass application-specific metadata in Arrow protocol messages. This includes `Field`, `Schema`, and `Message`.

The colon symbol `:` is to be used as a namespace separator. It can be used multiple times in a key.

The `ARROW` pattern is a reserved namespace for internal Arrow use in the `custom_metadata` fields. For example, `ARROW:extension:name`.

Extension Types

User-defined "extension" types can be defined setting certain `KeyValue` pairs in `custom_metadata` in the `Field` metadata structure. These extension keys are:

- `'ARROW:extension:name'` for the string name identifying the custom data type. We recommend that you use a "namespace"-style prefix for extension type names to minimize the possibility of conflicts with multiple Arrow readers and writers in the same application. For example, use `myorg.name_of_type` instead of simply `name_of_type`
- `'ARROW:extension:metadata'` for a serialized representation of the `ExtensionType` necessary to reconstruct the custom type

Note

Extension names beginning with `arrow.` are reserved for canonical extension types, they should not be used for third-party extension types.

This extension metadata can annotate any of the built-in Arrow logical types. The intent is that an implementation that does not support an extension type can still handle the underlying data. For example a 16-byte UUID value could be embedded in `FixedSizeBinary(16)`, and implementations that do not have this extension type can still work with the underlying binary values and pass along the `custom_metadata` in subsequent Arrow protocol messages.

Extension types may or may not use the `'ARROW:extension:metadata'` field. Let's consider some example extension types:

- `uuid` represented as `FixedSizeBinary(16)` with empty metadata
- `latitude-longitude` represented as `struct<latitude: double, longitude: double>`, and empty metadata

- `tensor` (multidimensional array) stored as `Binary` values and having serialized metadata indicating the data type and shape of each value. This could be JSON like `{'type': 'int8', 'shape': [4, 5]}` for a 4x5 cell tensor.
- `trading-time` represented as `Timestamp` with serialized metadata indicating the market trading calendar the data corresponds to

See also

Canonical Extension Types

Implementation guidelines

An execution engine (or framework, or UDF executor, or storage engine, etc) can implement only a subset of the Arrow spec and/or extend it given the following constraints:

Implementing a subset of the spec

- **If only producing (and not consuming) arrow vectors**: Any subset of the vector spec and the corresponding metadata can be implemented.
- **If consuming and producing vectors**: There is a minimal subset of vectors to be supported. Production of a subset of vectors and their corresponding metadata is always fine. Consumption of vectors should at least convert the unsupported input vectors to the supported subset (for example Timestamp.millis to timestamp.micros or int32 to int64).

Extensibility

An execution engine implementor can also extend their memory representation with their own vectors internally as long as they are never exposed. Before sending data to another system expecting Arrow data, these custom vectors should be converted to a type that exist in the Arrow spec.